

CDAC PL/SQL Important Templates

1. Procedure with Select Into and If Else

```
DELIMITER $

CREATE PROCEDURE procedure_name
(
    IN p_id INT
)
BEGIN

    DECLARE value1 INT;

    SELECT column_name
    INTO value1
    FROM table_name
    WHERE condition;

    IF value1 > value THEN

        SELECT 'MESSAGE';

    ELSE

        SELECT 'MESSAGE';

    END IF;

END $

DELIMITER ;
```

Call

```
CALL procedure_name(1);
```

2. Procedure with Update

```
DELIMITER $

CREATE PROCEDURE procedure_name
(
    IN p_id INT,
    IN new_value INT
)
BEGIN

    UPDATE table_name
    SET column_name = new_value
    WHERE condition;

END $

DELIMITER ;
```

Call

```
CALL procedure_name(1,1000);
```

3. Procedure with Delete

```
DELIMITER $

CREATE PROCEDURE procedure_name
(
    IN p_name VARCHAR(50)
)
BEGIN

    DELETE FROM table_name
    WHERE column_name = p_name;

END $

DELIMITER ;
```

Call

```
CALL procedure_name( 'ABC' );
```

4. Procedure with Insert

```
DELIMITER $  
  
CREATE PROCEDURE procedure_name  
(  
    IN value1 datatype,  
    IN value2 datatype  
)  
BEGIN  
  
    INSERT INTO table_name  
    VALUES(value1, value2);  
  
END $  
  
DELIMITER ;
```

Call

```
CALL procedure_name(value1,value2);
```

5. Function

```
DELIMITER $  
  
CREATE FUNCTION function_name  
(  
    value1 INT,  
    value2 INT  
)  
RETURNS INT  
DETERMINISTIC  
  
BEGIN
```

```
DECLARE result INT;

SET result = value1 + value2;

RETURN result;

END $

DELIMITER ;
```

Call

```
SELECT function_name(10,20);
```

6. Transaction

```
START TRANSACTION;

UPDATE table_name
SET column_name = value
WHERE condition;

UPDATE table_name
SET column_name = value
WHERE condition;

COMMIT;
```

```
ROLLBACK;
```

7. Fund Transfer Procedure

```
DELIMITER $

CREATE PROCEDURE procedure_name
(
    IN p_from INT,
    IN p_to INT,
    IN p_amount INT
```

```

)
BEGIN

    DECLARE balance_amt INT;

    START TRANSACTION;

    SELECT balance
    INTO balance_amt
    FROM table_name
    WHERE condition;

    IF balance_amt >= p_amount THEN

        UPDATE table_name
        SET balance = balance - p_amount
        WHERE condition;

        UPDATE table_name
        SET balance = balance + p_amount
        WHERE condition;

        COMMIT;

    ELSE

        ROLLBACK;

    END IF;

END $

DELIMITER ;

```

Call

```
CALL procedure_name(101,102,5000);
```

8. Cursor

```

DECLARE done INT DEFAULT 0;

DECLARE id INT;

```

```

DECLARE cur CURSOR FOR
SELECT column_name
FROM table_name;

DECLARE CONTINUE HANDLER
FOR NOT FOUND
SET done = 1;

OPEN cur;

read_loop: LOOP

    FETCH cur INTO id;

    IF done = 1 THEN
        LEAVE read_loop;
    END IF;

END LOOP;

CLOSE cur;

```

Call

```
CALL procedure_name();
```

Cursor is usually written inside a procedure and then called using `CALL procedure_name();`

9. Trigger

```

DELIMITER $

CREATE TRIGGER trigger_name

AFTER INSERT
ON table_name

FOR EACH ROW

BEGIN

    UPDATE table_name

```

```
    SET column_name = value
    WHERE condition;

END $

DELIMITER ;
```

Call

```
INSERT INTO table_name VALUES(...);
```

Trigger executes automatically. No CALL statement is required.

10. While Loop

```
DECLARE i INT DEFAULT 1;

WHILE i <= n DO

    IF condition THEN

        SELECT i;

    END IF;

    SET i = i + 1;

END WHILE;
```

Call

```
CALL procedure_name();
```

Loop is usually written inside a procedure.

11. Repeat Loop

```
DECLARE i INT DEFAULT 1;

REPEAT
```

```
statements;  
  
SET i = i + 1;  
  
UNTIL condition  
  
END REPEAT;
```

Call

```
CALL procedure_name();
```

12. Exception Handler

```
DECLARE EXIT HANDLER  
FOR SQLEXCEPTION  
  
BEGIN  
  
ROLLBACK;  
  
END;
```

Call

```
CALL procedure_name();
```

Exception Handler is written inside a procedure.

13. Master Template

```
DELIMITER $  
  
CREATE PROCEDURE procedure_name  
(  
    IN p_id INT  
)  
BEGIN
```



```
DECLARE value1 INT;

SELECT column_name
INTO value1
FROM table_name
WHERE condition;

IF condition THEN

    UPDATE table_name
    SET column_name = value
    WHERE condition;

    INSERT INTO table_name
    VALUES(...);

    COMMIT;

ELSE

    ROLLBACK;

END IF;

END $

DELIMITER ;
```

Call

```
CALL procedure_name(1);
```

Assignment 2 — Date Functions

Current Date

```
SELECT CURRENT_DATE();
```

Year Filter

```
SELECT *  
FROM emp  
WHERE YEAR(hiredate)=1982;
```

Month Filter

```
SELECT *  
FROM emp  
WHERE MONTHNAME(hiredate)='DECEMBER';
```

Day Name

```
SELECT DAYNAME(hiredate)  
FROM emp;
```

Format Date

```
SELECT DATE_FORMAT(hiredate, '%d/%m/%Y')  
FROM emp;
```

Date Difference

```
SELECT DATEDIFF(date1,date2)  
FROM table_name;
```

Add Days

```
SELECT ADDDATE(CURDATE(),15);
```

Months Difference

```
SELECT TIMESTAMPDIFF(MONTH,date1,date2)  
FROM table_name;
```

Uppercase

```
SELECT UPPER(ename)
FROM emp;
```

Length

```
SELECT LENGTH(ename)
FROM emp;
```

Assignment 3 — WHERE / ORDER BY / LIKE

Salary Condition

```
SELECT *
FROM emp
WHERE sal > 3000;
```

BETWEEN

```
SELECT *
FROM emp
WHERE sal BETWEEN 1000 AND 2000;
```

LIKE

```
SELECT *
FROM emp
WHERE ename LIKE 'S%';
```

Second Letter

```
SELECT *
FROM emp
WHERE ename LIKE '_A%';
```

Contains

```
SELECT *  
FROM emp  
WHERE ename LIKE '%A%';
```

IN Operator

```
SELECT *  
FROM emp  
WHERE deptno IN(10,20);
```

NULL

```
SELECT *  
FROM emp  
WHERE comm IS NULL;
```

ORDER BY

```
SELECT *  
FROM emp  
ORDER BY sal DESC;
```

Assignment 4 — GROUP BY / HAVING / AGGREGATE

Count

```
SELECT COUNT(*) FROM emp;
```

Maximum and Minimum Salary

```
SELECT MAX(sal), MIN(sal)  
FROM emp;
```

Average Salary

```
SELECT AVG(sal)
FROM emp;
```

Total Salary

```
SELECT SUM(sal)
FROM emp;
```

Group By Department

```
SELECT deptno, COUNT(*), AVG(sal)
FROM emp
GROUP BY deptno;
```

Having Clause

```
SELECT deptno, COUNT(*)
FROM emp
GROUP BY deptno
HAVING COUNT(*) > 2;
```

Job Wise Salary

```
SELECT job, SUM(sal)
FROM emp
GROUP BY job;
```

Assignment 5 — JOINS

Inner Join

```
SELECT e.ename, d.dname
FROM emp e
```

```
JOIN dept d
ON e.deptno = d.deptno;
```

Left Join

```
SELECT d.dname, e.ename
FROM dept d
LEFT JOIN emp e
ON d.deptno = e.deptno;
```

Self Join

```
SELECT e1.ename employee,
       e2.ename manager
FROM emp e1,
     emp e2
WHERE e1.mgr = e2.empno;
```

Multiple Table Join

```
SELECT s.sname,
       c.cname,
       o.amount
FROM salesman s,
     customer c,
     orders o
WHERE s.sid = o.sid
     AND c.cid = o.cid;
```

Assignment 6 — SUBQUERY

Maximum Salary

```
SELECT *
FROM emp
WHERE sal =
(
    SELECT MAX(sal)
```

```
FROM emp  
);
```

Less Than Average Salary

```
SELECT *  
FROM emp  
WHERE sal <  
(  
    SELECT AVG(sal)  
    FROM emp  
);
```

Department Average Salary

```
SELECT *  
FROM emp  
WHERE sal >  
(  
    SELECT AVG(sal)  
    FROM emp  
    WHERE deptno = 20  
);
```

EXISTS

```
SELECT *  
FROM emp e  
WHERE EXISTS  
(  
    SELECT *  
    FROM dept d  
    WHERE e.deptno = d.deptno  
);
```

Second Highest Salary

```
SELECT MAX(sal)  
FROM emp  
WHERE sal <  
(
```

```
SELECT MAX(sal)
FROM emp
);
```

Correlated Subquery

```
SELECT *
FROM emp e1
WHERE sal >
(
  SELECT AVG(sal)
  FROM emp e2
  WHERE e1.deptno = e2.deptno
);
```
